ASSIGNMENT: Straight Lane Detection

Date: 2025-APRIL-07

Author: HWANG MooYeol 22000814

Github: [-]

Demo Video: [-]

Introduction

1. Objective

- **Goal**: Detect lanes and vanishing point

  In this assignment, detect and draw lanes (left and right each) on the given images. Also, find and draw the vanishing point, the point where the two lane lines intersect.

2. Preparation

Software Installation

- OpenCV 4.9.0, Visual Studio 2022

Algorithm

1. Overview

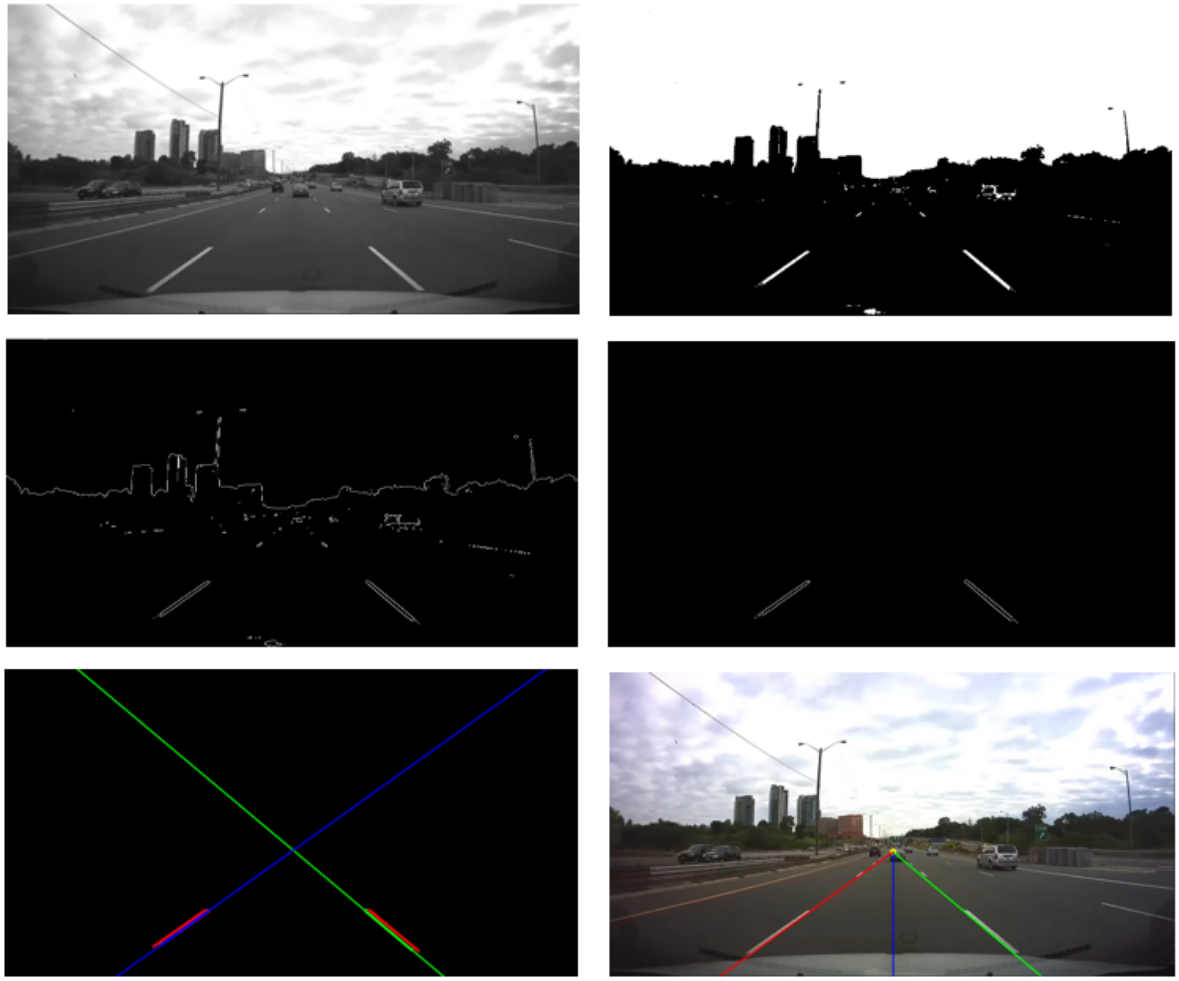


Figure 1. Overview

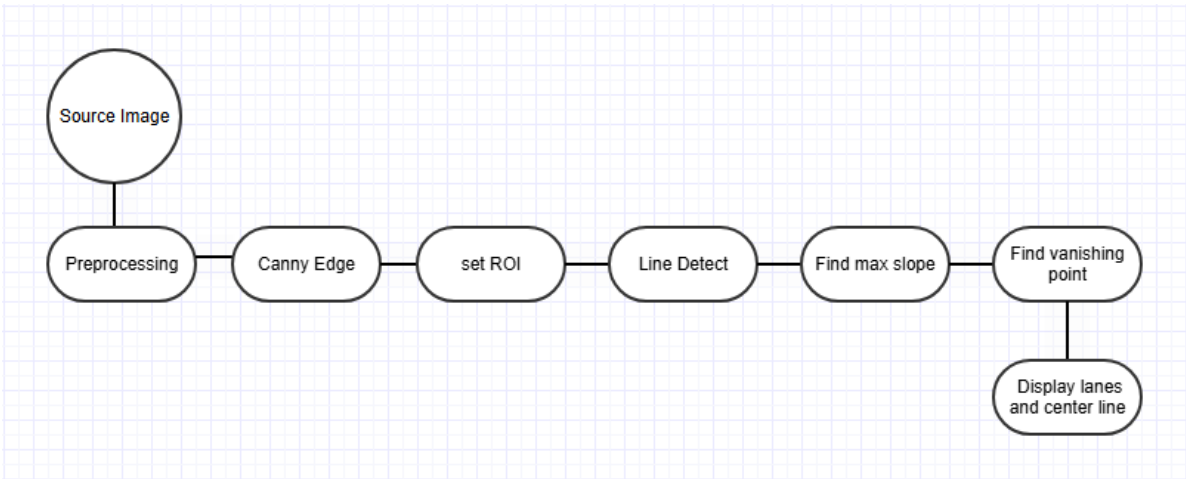


Figure 2. Flowchart

2. Procedure

Convert Grayscale and Apply filter

Figure 3. Grayscale and Filter

```
cvtColor(src, gray_src, COLOR_BGR2GRAY);

blur(gray_src, gray_src, Size(3, 3), Point(-1, -1))
```

The blur filter and cvtColor was used for image preprocessing for thresholding.

Threshold and Morphology

Figure 4. Threshold

```
threshold(gray_src, imthres, 127, 255, THRESH_BINARY);
```

The threshold function was used to separate the background and lane. The line was detected to be clean without using additional closing morphology.

Canny Edge detection

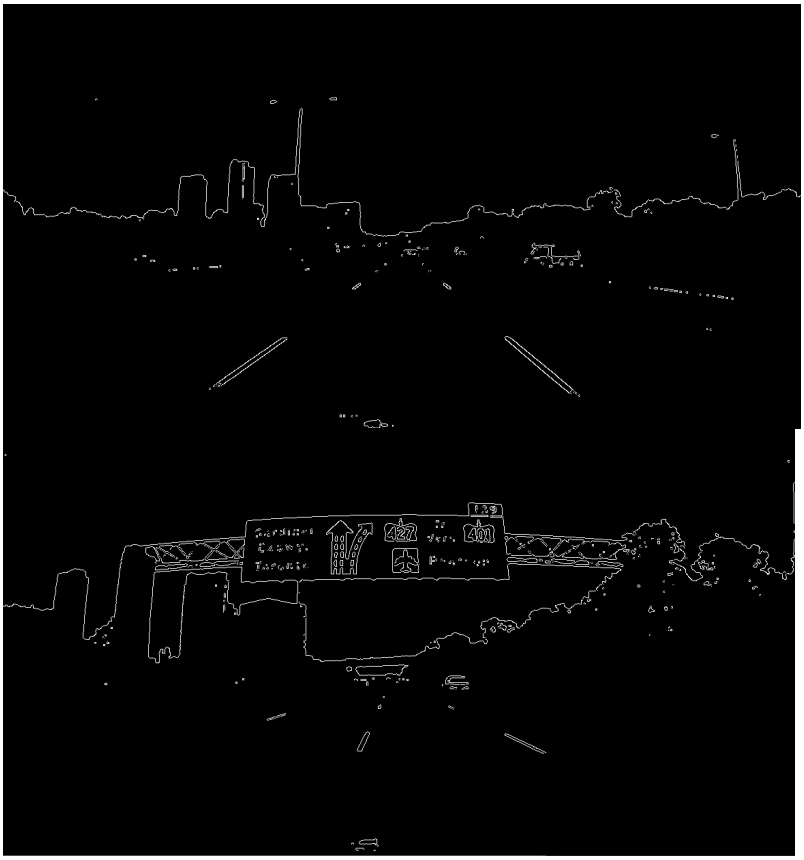


Figure 5. Detect Edge

The `Canny` function was used for edge detection

```
void cv::Canny  (   InputArray  image,
                    OutputArray     edges,
                    double  threshold1,
                    double  threshold2,
                    int     apertureSize = 3,
                    bool    L2gradient = false
                )
```

Set ROI and mask

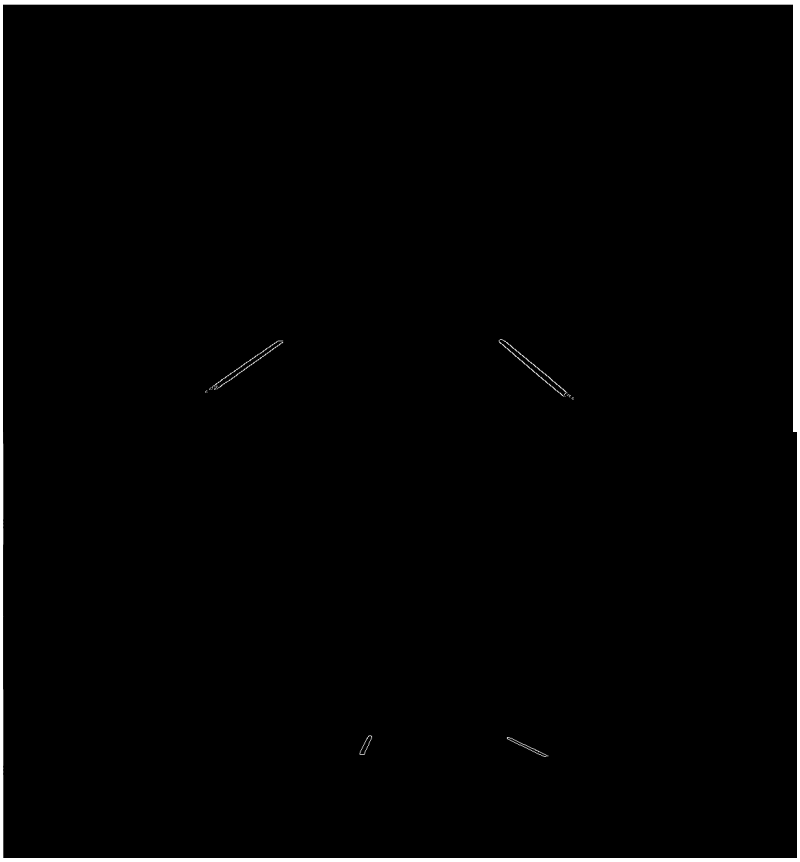


Figure 6. ROI and Mask

```
// Define ROI rectangle
Rect roi(300, 440, 550, 160);

// Create masks
Mat mask = Mat::zeros(dst.size(), dst.type());
dst(roi).copyTo(mask(roi));

// Convert for display
cvtColor(mask, cdst, COLOR_GRAY2BGR);
```

The ROI was calculated and set. Masked the original image for coordinate value unification.

HoughLinesP

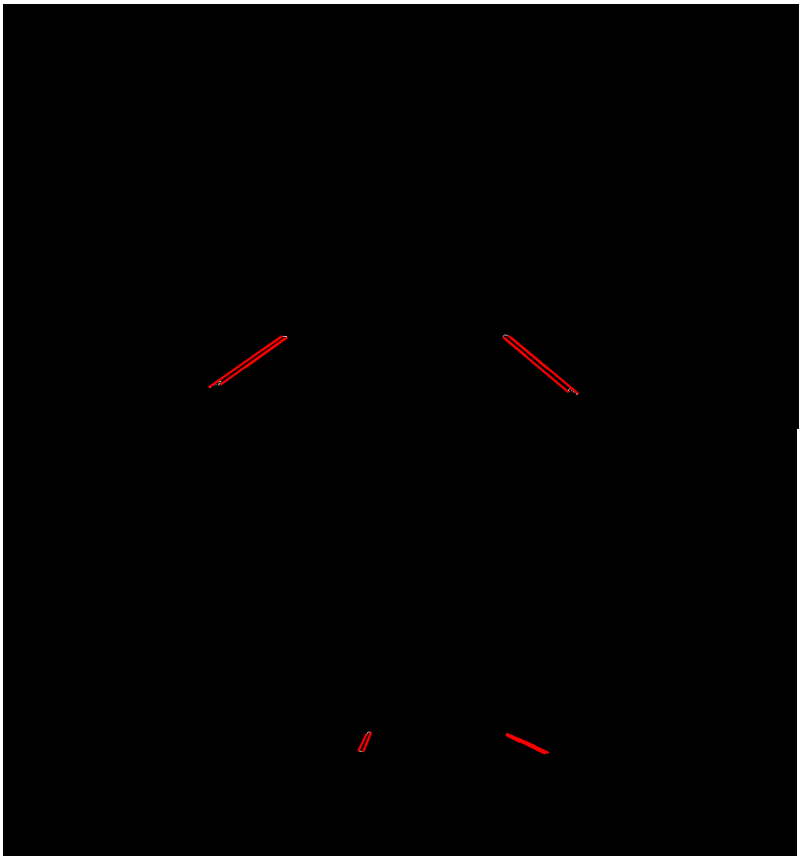


Figure 7. detect line

The `HoughLinesP` function was used for Line detection

```
void cv::HoughLinesP    (   InputArray  image,
                            OutputArray     lines,
                            double  rho,
                            double  theta,
                            int     threshold,
                            double  minLineLength = 0,
                            double  maxLineGap = 0
                        )
```

Also, function `HoughLines` can be used for Line detection

```
void cv::HoughLines (   InputArray  image,
                        OutputArray     lines,
                        double  rho,
                        double  theta,
                        int     threshold,
                        double  srn = 0,
                        double  stn = 0,
                        double  min_theta = 0,
                        double  max_theta = CV_PI
                    )
```

Validate Best Line for Lane

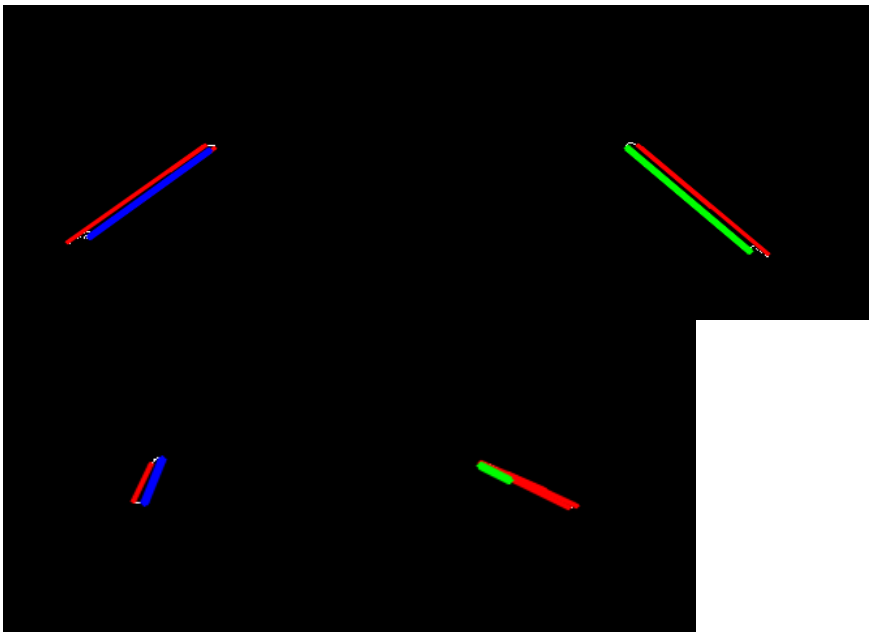


Figure 8. detect lane

For the algorithm for selecting one line per lane, I used slope comparator.

```
// Find the line with the largest slope for linesP
Vec4i maxPositiveLine1, maxNegativeLine1;
double maxPositiveSlope1 = 0, maxNegativeSlope1 = 0;
bool foundPositive1 = 0, foundNegative1 = 0;

for (size_t i = 0; i < linesP.size(); i++) {
    Vec4i l = linesP[i];
    int dx = l[2] - l[0];
    int dy = l[3] - l[1];

    double slope = (double)dy / dx;

    if (slope > 0) {
        if (!foundPositive1 || slope > maxPositiveSlope1) {
            maxPositiveSlope1 = slope;
            maxPositiveLine1 = l;
            foundPositive1 = 1;
        }
    }
    else if (slope < 0) {
        if (!foundNegative1 || abs(slope) > abs(maxNegativeSlope1)) {
            maxNegativeSlope1 = slope;
            maxNegativeLine1 = l;
            foundNegative1 = 1;
        }
    }
}

if (foundPositive1)
    line(cdst, Point(maxPositiveLine1[0], maxPositiveLine1[1]),
Point(maxPositiveLine1[2], maxPositiveLine1[3]), Scalar(0, 255, 0), 4, LINE_AA);
```

```
if (foundNegative1)
    line(cdst, Point(maxNegativeLine1[0], maxNegativeLine1[1]),
Point(maxNegativeLine1[2], maxNegativeLine1[3]), Scalar(255, 0, 0), 4, LINE_AA);
```

In order to select one lane from the left lines, the highest slope among the positive slope values was selected by comparing, and the right lane was selected by comparison with a negative slope.

Calculate vanishing point

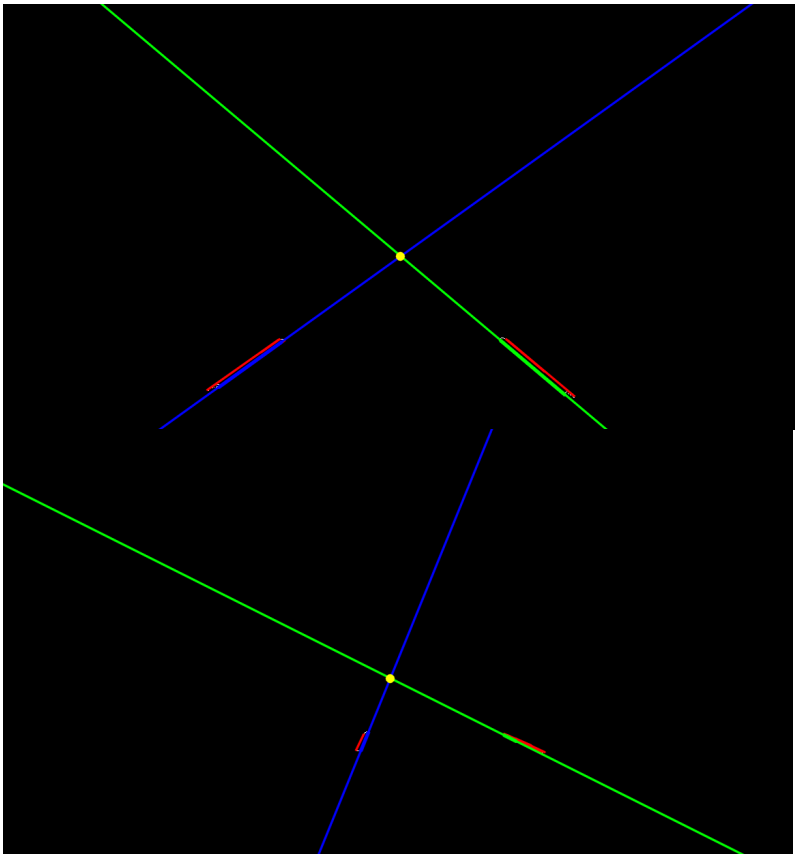
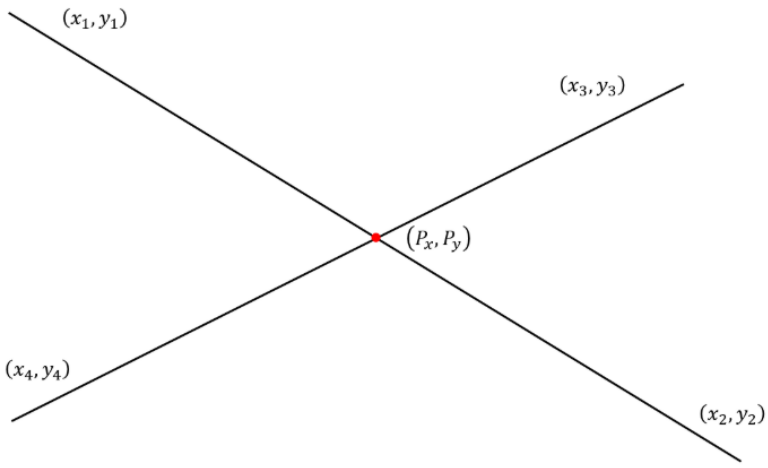


Figure 10. Find vanishing point

Since the vanishing point is the intersection of both lanes, Four points were selected from each line, and the vanishing point was calculated by the equation (cross-point equation) below.



두 점 (x1, y1), (x2, y2)를 지나는 직선 A와 (x3, y3), (x4, y4)를 지나는 직선 B의 교차점 P는 다음과 같다.

$$(P_x, P_y) = (\frac{(x_1y_2 - y_1x_2)(x_3 - x_4) - (x_1 - x_2)(x_3y_4 - y_3x_4)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)}, \frac{(x_1y_2 - y_1x_2)(y_3 - y_4) - (y_1 - y_2)(x_3y_4 - y_3x_4)}{(x_1 - x_2)(y_3 - y_4) - (y_1 - y_2)(x_3 - x_4)})$$

Figure 11. cross-point equation

```
float x1 = maxPositiveLine1[0], y1 = maxPositiveLine1[1], x2 =
maxPositiveLine1[2], y2 = maxPositiveLine1[3];
float x3 = maxNegativeLine1[0], y3 = maxNegativeLine1[1], x4 =
maxNegativeLine1[2], y4 = maxNegativeLine1[3];

float px = ((x1 * y2 - y1 * x2) * (x3 - x4) - (x1 - x2) * (x3 * y4 - y3 * x4)) /
((x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4));
float py = ((x1 * y2 - y1 * x2) * (y3 - y4) - (y1 - y2) * (x3 * y4 - y3 * x4)) /
((x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4));
Point2f intersection(px, py);
circle(cdst, intersection, 6, Scalar(0, 255, 255), -1, LINE_AA);
```

Display lanes

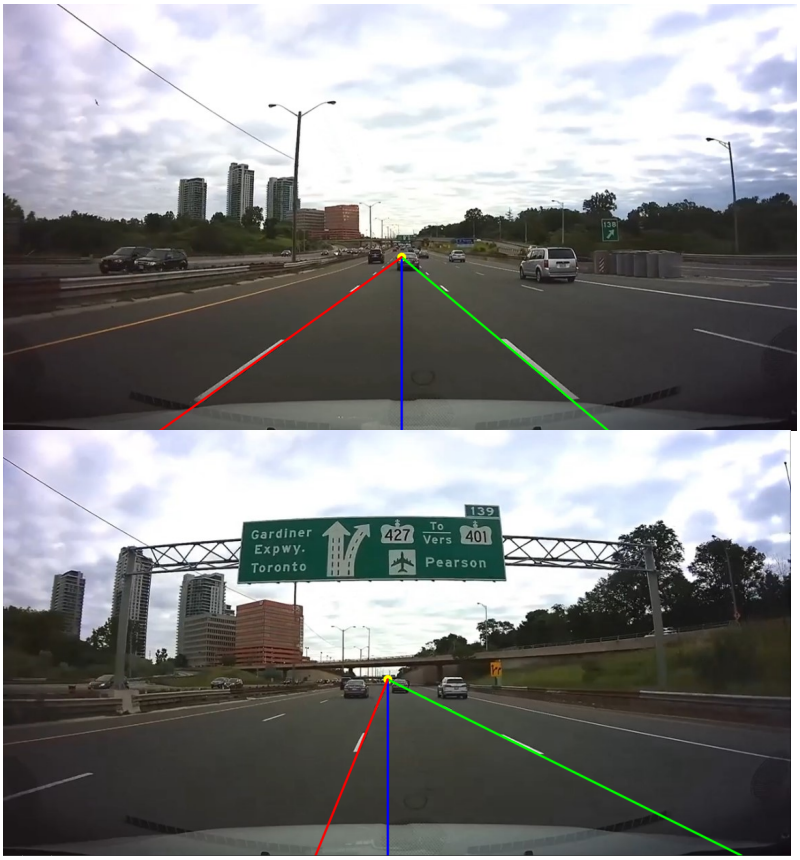


Figure 12. Display lanes and center line

Lane lines can be marked based on the vanishing point and the slope of each lane. In the case of the center line, a vertical line is drawn to the bottom of the screen based on the vanishing point.

```
//centerline
line(src, intersection, Point(px, src.rows), Scalar(255, 0, 0), 2, LINE_AA);

// Positive line
double slopePos = (double)(maxPositiveLine1[3] - maxPositiveLine1[1]) /
(maxPositiveLine1[2] - maxPositiveLine1[0]);
double x_bottom_pos = (src.rows - py) / slopePos + px;
line(src, intersection, Point((int)x_bottom_pos, src.rows), Scalar(0, 255, 0),
2, LINE_AA);

// Negative line
double slopeNeg = (double)(maxNegativeLine1[3] - maxNegativeLine1[1]) /
(maxNegativeLine1[2] - maxNegativeLine1[0]);
double x_bottom_neg = (src.rows - py) / slopeNeg + px;
line(src, intersection, Point((int)x_bottom_neg, src.rows), Scalar(0, 0, 255),
2, LINE_AA);
```

Result and Discussion

1. Final Result

In each pictures. The picture on the left is the result of the original picture showing defective teeth. The picture in the center is the result of showing the area values of the teeth. The picture on the right is the result of the analysis of the gear.

Lane center

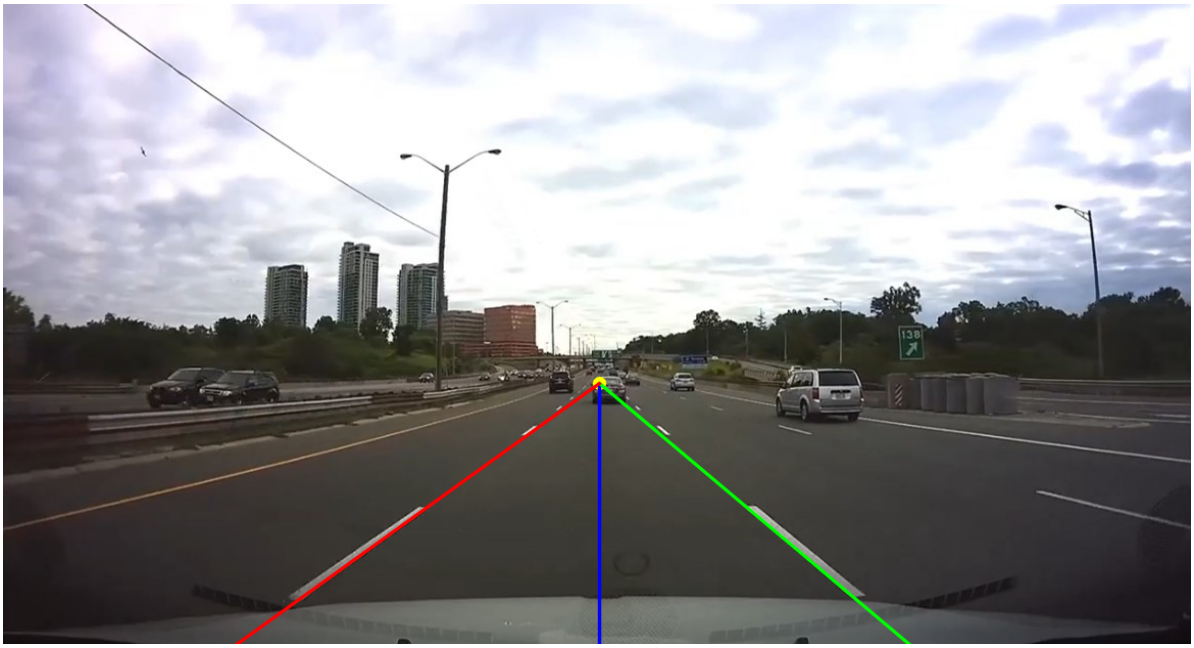


Figure 13. result for Lane Center

Lane changing

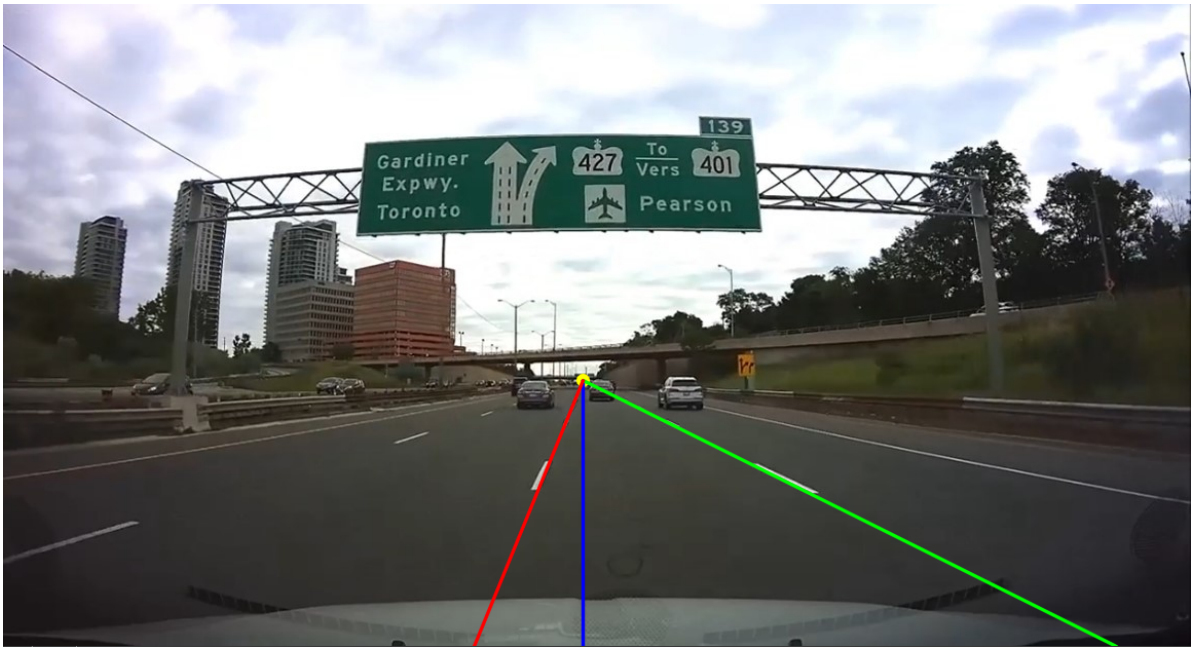


Figure 14. result for Changing Lane

2. Discussion

From the results, it can be seen that the lane and vanishing point were accurately determined. In addition, it is possible to determine whether the car is maintaining the lane or changing the lane through the relationship between the center line and the lane. The proposed algorithm has achieved the project goal successfully.

However, in order to process lanes through videos rather than images, an algorithm that can process the ROI area more fluidly seems to be needed. Also, HoughLines may have less computation than HoughLinesP.

Conclusion

This project aims to develop a machine vision system for detecting straight lanes and calculating the vanishing point in road images. By applying image processing techniques including grayscale conversion, filtering, thresholding, edge detection, region of interest (ROI) masking, and Hough transform the system successfully identifies the left and right lanes and accurately estimates their intersection, the vanishing point.

While this image processing approach is effective for lane detection, it has several limitations when applied to video processing.

One major issue is that the algorithm relies on static thresholds and fixed ROI settings, which may not work well when the camera view or lighting conditions change dynamically in a video. For instance, shadows, sudden brightness changes, or poor weather can affect edge and line detection accuracy.

Therefore, for reliable video processing, it's important to consider enhancements such as adaptive thresholding**,** dynamic ROI adjustment.

In conclusion, the proposed algorithm effectively achieves its goal in detecting lanes and calculating the vanishing point but other algorithms are recommended for real-time video processing.

Appendix

```
/* -----------------------------------------------------
 * Lane detection
 * Created : 2025.04.02
 *----------------------------------------------------- */

#include "opencv2/imgcodecs.hpp"
#include "opencv2/highgui.hpp"
#include "opencv2/imgproc.hpp"

using namespace cv;
```

```
using namespace std;

void processLaneImage(const string& windowName, const char* filename) {
    Mat gray_src, imthres, dst, cdst;

    Mat src = imread(filename);
    if (src.empty()) {
        printf(" Error opening image: %s\n", filename);
        return;
    }

    cvtColor(src, gray_src, COLOR_BGR2GRAY);
    blur(gray_src, gray_src, Size(3, 3), Point(-1, -1));
    threshold(gray_src, imthres, 127, 255, THRESH_BINARY);
    Canny(imthres, dst, 50, 300, 3);

    Rect roi(300, 440, 550, 160);
    Mat mask = Mat::zeros(dst.size(), dst.type());
    dst(roi).copyTo(mask(roi));
    cvtColor(mask, cdst, COLOR_GRAY2BGR);

    vector<Vec4i> linesP;
    HoughLinesP(mask, linesP, 1, CV_PI / 180, 10, 15, 5);
    for (size_t i = 0; i < linesP.size(); i++) {
        Vec4i l = linesP[i];
        line(cdst, Point(l[0], l[1]), Point(l[2], l[3]), Scalar(0, 0, 255), 2,
LINE_AA);
    }

    Vec4i maxPositiveLine1, maxNegativeLine1;
    double maxPositiveSlope1 = 0, maxNegativeSlope1 = 0;
    bool foundPositive1 = 0, foundNegative1 = 0;

    for (size_t i = 0; i < linesP.size(); i++) {
        Vec4i l = linesP[i];
        int dx = l[2] - l[0];
        int dy = l[3] - l[1];

        double slope = (double)dy / dx;

        if (slope > 0) {
            if (!foundPositive1 || slope > maxPositiveSlope1) {
                maxPositiveSlope1 = slope;
                maxPositiveLine1 = l;
                foundPositive1 = 1;
            }
        }
        else if (slope < 0) {
            if (!foundNegative1 || abs(slope) > abs(maxNegativeSlope1)) {
                maxNegativeSlope1 = slope;
                maxNegativeLine1 = l;
                foundNegative1 = 1;
            }
        }
    }

    if (foundPositive1)
```

```
        line(cdst, Point(maxPositiveLine1[0], maxPositiveLine1[1]),
Point(maxPositiveLine1[2], maxPositiveLine1[3]), Scalar(0, 255, 0), 4, LINE_AA);

    if (foundNegative1)
        line(cdst, Point(maxNegativeLine1[0], maxNegativeLine1[1]),
Point(maxNegativeLine1[2], maxNegativeLine1[3]), Scalar(255, 0, 0), 4, LINE_AA);

    if (foundPositive1 && foundNegative1) {
        float x1 = maxPositiveLine1[0], y1 = maxPositiveLine1[1], x2 =
maxPositiveLine1[2], y2 = maxPositiveLine1[3];
        float x3 = maxNegativeLine1[0], y3 = maxNegativeLine1[1], x4 =
maxNegativeLine1[2], y4 = maxNegativeLine1[3];

        float px = ((x1 * y2 - y1 * x2) * (x3 - x4) - (x1 - x2) * (x3 * y4 - y3
* x4)) /
            ((x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4));
        float py = ((x1 * y2 - y1 * x2) * (y3 - y4) - (y1 - y2) * (x3 * y4 - y3
* x4)) /
            ((x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4));
        Point2f intersection(px, py);

        circle(cdst, intersection, 6, Scalar(0, 255, 255), -1, LINE_AA);
        circle(src, intersection, 6, Scalar(0, 255, 255), -1, LINE_AA);
        line(src, intersection, Point(px, src.rows), Scalar(255, 0, 0), 2,
LINE_AA);

        double slopePos = (double)(maxPositiveLine1[3] - maxPositiveLine1[1]) /
(maxPositiveLine1[2] - maxPositiveLine1[0]);
        double x_bottom_pos = (src.rows - py) / slopePos + px;
        line(src, intersection, Point((int)x_bottom_pos, src.rows), Scalar(0,
255, 0), 2, LINE_AA);

        double slopeNeg = (double)(maxNegativeLine1[3] - maxNegativeLine1[1]) /
(maxNegativeLine1[2] - maxNegativeLine1[0]);
        double x_bottom_neg = (src.rows - py) / slopeNeg + px;
        line(src, intersection, Point((int)x_bottom_neg, src.rows), Scalar(0, 0,
255), 2, LINE_AA);
    }

    imshow(windowName, src);
}

int main(int argc, char** argv)
{
    processLaneImage("Lane Center", "Lane_center.JPG");
    processLaneImage("Lane Changing", "Lane_changing.JPG");

    waitKey();
    return 0;
}
```